Experiment No -09 (Group C)

Title:-

Implementation of Hash Table using Division Method and Linear Probing

Problem statement:-

Design and implement a hash table of fixed size. Use the **division method** for the hash function and resolve **collisions** using **linear probing**. Allow the user to perform the following **operations**:

•Insert a key

•Search for a key

•Delete a key

•Display the table

Pre-requisite:-

- Basic knowledge of **Python programming**
- Understanding of **Variables, lists, loops, and conditional statements.**
- Basic knowledge of **Functions and classes in Python.**
- Basic concepts of hashing and hash functions.
- Basic knowledge of **collision handling.**

Objective:-

- To design and implement a **fixed-size hash table**.
- To implement the **division method** for calculating the hash index.
- To handle collisions using **linear probing**.
- To perform the following operations: Insert a key, Search for a key, Delete a key, Display the hash table

Software and Hardware requirements:-

Software :-

Operating System recommended:- 64-bit Open source Linux or Windows or macOS

Programming tools recommended: - Python 3.x

Programming IDE/Editor: IDLE, VS Code, PyCharm, or any Python-supported editor

Hardware :-

Processor: Intel Core i3 or equivalent and above

RAM: Minimum 2 GB

Storage: Minimum 100 MB free space

Keyboard and Monitor: Required for entering input and viewing output.

Outcome:-

After completing this experiment, students will be able to:

- Understand the concept of hash tables.
- Implement a hash function using the division method.
- Understand and implement linear probing for collision resolution.
- Perform insertion, searching, deletion, and display operations on a hash table.
- Understand how collisions occur and how they can be resolved.
- Develop a basic hash table program using Python.

Theory:-

Introduction to Hashing

Hashing is the technique of mapping a large chunk of data into small tables using a hash function.

A hash table is a data structure that provides fast insertion, deletion, and searching operations using a key.

It uses a hash function to compute an index into an array of slots, from which the desired value can be found.

Division Method

This is the most simple and easiest method to generate a hash value for any data item k which you want top store in the hash table.

The hash function divides the value k by M and then uses the remainder obtained.

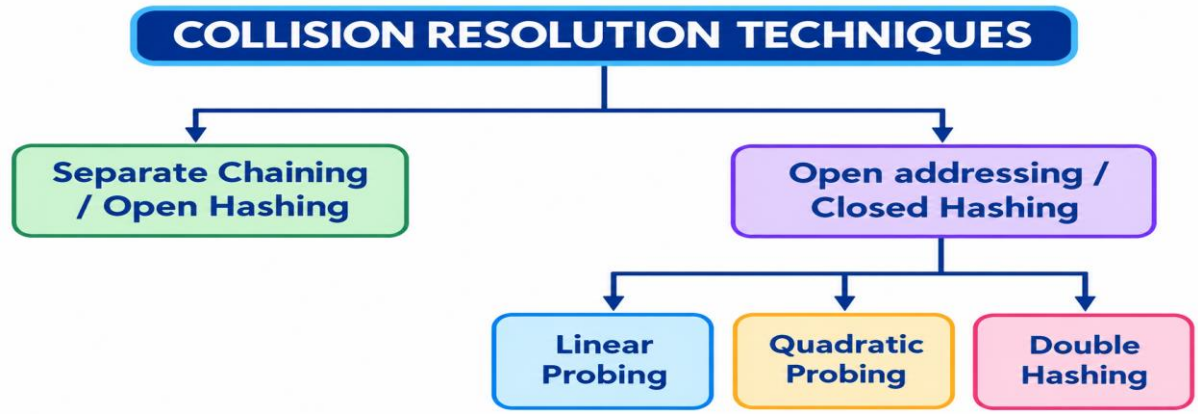
h(K) = k mod M Here, k is the key value, and M is the size of the hash table.

It is best suited that M is a prime number as that can make sure that keys are more uniformly distributed.

Example: k = 12345 , M = 95 h(12345) = 12345 mod 95= 90

k = 1276, M = 11 h(1276) = 1276 mod 11 = 0

Collision resolution strategies:-



Linear Probing

The idea behind Linear Probing simple we a fixed sized hash table & whenever collision occurs we traverse linearly / Sequential in cyclic manner to find next empty slot.

It mean when we want to insert key & if a key is mapped (hashed) to an index which is already occupied then check for next available position to place them. i.e.

H(key) = h, h(k)+1,h(k)+2,h(k)+3,……..,h(k)+I

Example-

let us consider a simples= hash function as key % table Size which means,

Hash(key)=key % S where S→ size of the table.

Let us consider S=7 & index value in table are form 0 to 6.

Now if we want to insert following value in table

(50,700,76,85,92,73,101)

Index / hash key is calculated for every key as follows

50 % 7 = 1

700 %7 = 0

76 % 7 = 6

85 % 7 =1

92% 7 =1

73 %7=3

101 % 7 =3

1) When we try to insert key ‘50’ it is inserted at index position 1.

2) Similarly key ‘700’ is inserted at index position 0.

3) Similarly key ‘76’ is inserted at index position 6 as shown in the hash table step 3.

0	700
1	50
2	
3	
4	
5	
6	76

4) When we try to insert ‘85’ since position calculated for ‘85’ is 1 collision occurs as ‘50’ is already present at position 1 so we store ‘85’ at next empty position i.e. at index position 2.

0	700
1	50
2	85
3	
4	
5	
6	76

5) When we try to insert ‘92’ since position calculated for ‘92’ is 1 again collision occurs as ‘50’ is already present at position 1 so we store ‘92’ at next empty position i.e. at index position 3

0	700
1	50
2	85
3	92
4	
5	
6	76

6) When we try to insert ‘73’ since position calculated for ‘92’ is 3 collision occurs as ‘92’ is already present at position 3 so we store ‘73’ at next empty position i.e. at index position 4.

0	700
1	50
2	85
3	92
4	73
5	
6	76

7) Finally while inserting ‘101’ since index position ‘3’ is already occupied to avoid collision we need to insert to insert at next empty position i.e.5.

0	700
1	50
2	85
3	92
4	73
5	101
6	76

Key	Location /Position	Probe
50	50 % 7 = 1	1
700	700 %7 = 0	1
76	76 % 7 = 6	1
85	85 % 7 =1	2
92	92% 7 =1	3
73	73 %7=3	2
101	101 % 7 =3	3
	Total Probes	13

Algorithm: Hash Table Using Division Method and Linear Probing

Step 1: Create Hash Table

Start.

Read the size of the hash table from the user.

Create a list of the given size and initialize all positions with None.

Step 2: Hash Function

Take the key as input.

Calculate the index using the division method:

index = key % size

Return the calculated index.

Step 3: Insert a Key

Read the key to be inserted.

Calculate the index using the hash function.

Check whether the calculated position is empty.

If the position is occupied, move to the next position using:

index = (index + 1) % size

Continue until an empty position is found.

Insert the key at the empty position.

Display **"Key inserted successfully.“**

Step 4: Search for a Key

Read the key to be searched.

Calculate its index using the hash function.

Check the current position.

If the key is found, display **"Key found."**

Otherwise, move to the next position using linear probing:

index = (index + 1) % size

Repeat the process for all positions in the table.

If the key is not found, display **"Key not found.“**

Step 5: Delete a Key

Read the key to be deleted.

Calculate its index using the hash function.

Check the current position.

If the key is found, replace it with "DELETED".

Display **"Key deleted."**

If the key is not found after checking the table, display **"Key not found."**

Step 6: Display the Hash Table

Traverse all positions of the hash table.

Display the index and the value stored at each position.

Display None for empty positions.

Step 7: Menu

Display the following menu:

1. Insert 2. Search 3. Delete 4. Display 5. Exit

Read the user's choice.

Perform the corresponding operation.

Repeat the menu until the user selects **Exit**.

Stop.

Conclusion:-

The hash table was successfully designed and implemented using the **division method** as the hash function and **linear probing** for collision resolution.

The program allows the user to insert, search, delete, and display keys in the hash table.

This experiment helps in understanding the practical implementation of hashing and collision handling techniques.

Viva Questions:-

1. Which hash function is used in this program?
2. What is a hash table?
3. What is hashing?
4. What is the formula for the division method?
5. What is a collision?
6. How are collisions resolved in this program?
7. What is linear probing?
8. What is the formula for linear probing?
9. What is the load factor?
10. Can duplicate keys be inserted?